WOLI OS

Google AI Studio — Bug Fix & Improvement Prompts

Purpose

A phased prompt pack to repair the issues identified during the WOLI OS source-code audit. Paste the prompts into Google AI Studio one at a time, then verify each phase before moving on.

Core rule

Fix correctness and data integrity first. Preserve the existing application, visual design, architecture, user data and working features. Do not rebuild the app or make unrelated design changes.

Recommended order

Phase	Focus	Priority
1	Database initialization and backup/restore safety	Critical
2	Task, habit and goal relationship consistency	Critical
3	Goal progress and actual-time calculations	Critical
4	Daily Planner validation and overlap prevention	High
5	Analytics calculation correctness	High
6	Time Tracker manual-entry and session integrity	High
7	Calendar, History and Settings verification	Medium
8	Full regression testing and release-readiness audit	Critical

How to use: Open the existing WOLI OS project in Google AI Studio. Paste the Global Instruction first. Then paste Phase 1, review the changes and test them. Continue one phase at a time. Do not submit all phases together.

These prompts are based on source-code findings. Browser interactions, responsive behavior and offline workflows still require testing in a running app.

GLOBAL INSTRUCTION — Paste this first

Apply this constraint throughout the repair session.

```
You are working on my EXISTING WOLI OS application. Repair the specific defects I describe. Do NOT rebuild the app.

MANDATORY RULES
1. Inspect the existing code and trace the root cause before editing. First identify relevant files, data models, functions and call paths.
2. Preserve the current UI, layout, colors, typography, navigation, component structure, features and user experience unless a phase explicitly requests a small control.
3. Preserve the current stack, React/TypeScript architecture, project structure and IndexedDB/Dexie database. Do not migrate frameworks or replace the database.
4. Never delete, reset, reseed, overwrite or silently modify existing user data. Never use destructive database operations as a shortcut.
5. If a schema change is needed, use a safe versioned migration that preserves all existing records and relationships.
6. Avoid duplicate sources of truth. Reuse existing shared context, utilities, types and database services.
7. Do not add fake records, hard-coded progress, placeholder success states or fabricated analytics.
8. Limit changes to the current phase. Do not refactor unrelated modules or add unrequested features.
9. Add appropriate validation, loading, empty, success and error states using the existing design language.
10. Run available type checks, lint/build commands and relevant tests. If a command cannot run, state why. Never claim an unexecuted test passed.

WORKFLOW
A. Explain the root cause and files to be changed.
B. Implement the smallest safe fix.
C. Explain each changed file.
D. List tests actually executed, their results and tests still requiring manual browser verification.
E. Do not proceed until the current phase is complete and data-safety checks pass.

If an issue is already fixed in the current code, do not rewrite it. Show evidence and test it.
```

PHASE 1 — Database initialization and backup/restore safety

Audit issue: starter data may be inserted again when tables are empty, including after a user intentionally wipes data or restores an empty backup.

```
Inspect database initialization and Data & Settings backup/restore code. Find why starter records can be
reappear after a wipe or empty restore.

REQUIREMENTS
1. Add a persistent first-run/seed-completed marker using the existing database/settings mechanism. Seed
starter data only during intended first-time onboarding.
2. Never use “tasks table is empty” or “habits table is empty” as the only condition for seeding.
3. After a user deletes all tasks and habits, refresh/reopen must leave the database empty.
4. Restoring a valid backup containing zero records must remain empty after reload.
5. Validate JSON structure, supported schema/version, required tables and record shapes before modifying
existing data.
6. Show a restore preview with counts by entity type before confirmation.
7. Make Replace and Merge behavior explicit. Replace requires confirmation. Merge must avoid duplicate IDs
and define conflict handling.
8. If validation fails, leave the existing database unchanged and show a useful error.
9. Use a safe transaction for restore where supported. Roll back on failure; do not leave partially replaced
data.
10. Preserve existing records and use a safe migration if metadata is added.

TESTS
- Fresh first launch seeds/onboards only once as intended.
- Delete all tasks and habits, refresh and reopen: no starter data returns.
- Export an empty database, restore it, refresh: still empty.
- Malformed JSON and unsupported backup versions are rejected safely.
- Valid backup restores expected records and relationships.
- Failed restore does not partially destroy existing data.

Do not redesign Settings. Add only the minimum UI needed for a restore preview and confirmation.
```

PHASE 2 — Task, habit and goal relationship consistency

Audit issue: task/goal and habit/goal links can be stored on both sides. Updating only one side can leave stale references and incorrect progress.

```
Inspect task, habit and goal models and every create, edit, link, unlink, archive and delete flow. Trace
shared context and database operations.

REQUIREMENTS
1. Define one canonical relationship rule for task-goal links and one for habit-goal links. Use stable IDs,
never titles as identifiers.
2. When a task is linked to Goal A, ensure both sides agree if both sides are stored.
3. When a task moves from Goal A to Goal B, remove it from A and add it to B in one database transaction.
4. Unlinking must remove stale references from both sides.
5. Apply the same behavior to habits, whether linked from the Goal form or Habit form.
6. Deleting or archiving linked records must clean up active references according to a documented rule
without destroying historical records.
7. Deleting a goal must not leave active tasks/habits pointing to a nonexistent goal.
8. Prevent duplicate IDs in linkedTaskIds and linkedHabitIds.
9. Reconcile legacy one-sided links safely. Never infer relationships from matching titles.
10. Make multi-record relationship updates atomic using Dexie transactions or the existing transaction
abstraction.
11. Keep existing UI and category controls. Do not rebuild forms unnecessarily.

TESTS
- Link a task to Goal A; verify both records.
- Move it to Goal B; verify Goal A no longer references it.
- Unlink it; verify both sides are cleared.
- Link a habit from Habit interface; verify Goal sees it.
- Link the same habit from Goal interface; verify no duplicate.
- Archive/delete linked records; verify documented cleanup.
- Refresh/reopen; links persist.
- Two tasks with identical titles must remain independently linkable.

Report the exact files changed and the relationship rules implemented.
```

PHASE 3 — Goal progress and actual-time calculations

Audit issue: automatic goal progress can disagree with completed tasks and time records. Count eligible actual work only, and never count the same work twice.

```
Inspect the centralized goal-progress engine, time-session model, task completion flow, timer flow and every screen displaying goal progress: Dashboard, Goals, Goal Inspector and Analytics.

REQUIREMENTS
1. Use one authoritative goal-progress calculation service across all screens. Do not duplicate formulas in components.
2. Time-based goals count eligible recorded ACTUAL time only. Never count planned schedule duration as completed work.
3. Example: 90 minutes of eligible saved actual time against a 15-hour goal = 1.5/15 hours, 10% progress and 13.5 hours remaining.
4. If a task is merely marked complete but has no eligible actual-time record, do not invent time. Respect the selected measurement method.
5. Define eligibility for manual entries, timer sessions, paused sessions, deleted sessions and edited sessions.
6. Prevent double counting when a session references both a task and a goal, or when a task is linked to a goal. One unit of work contributes only once to a given goal.
7. Editing/deleting a session must recalculate affected task totals and goal progress.
8. Linking, unlinking or moving a task/habit must recalculate affected goals.
9. Recalculate after task completion, timer stop, manual entry save, session edit/delete, goal target change and measurement-method change.
10. Respect goal start date, deadline and horizon. Do not count work outside the valid period unless existing product rules explicitly permit it.
11. Keep units consistent. Avoid rounding before aggregation; round only for display.
12. Derive current progress, remaining target, percentage and milestones from the same calculation result.
13. Preserve manual progress when the selected method is manual. Do not overwrite it with automatic values.
14. The Goal category dropdown already exists in the inspected source; preserve and verify it rather than recreating it.

TESTS
- 90 eligible actual minutes / 15-hour goal = 1.5/15, 10%, 13.5 remaining.
- Planned 90 minutes without actual work does not increase an automatic time goal.
- One session linked to task and goal is counted once.
- Edit 90 minutes to 60 minutes: progress becomes 1/15 hours.
- Delete session: its contribution is removed.
- Move task between goals: both goals recalculate correctly.
- Dashboard, Goals, Inspector and Analytics show identical progress.
- Refresh/reopen preserves the result.

If the model cannot distinguish planned and actual time, explain the gap and implement the smallest safe migration. Do not fabricate historical actual time.
```

PHASE 4 — Daily Planner validation and overlap prevention

Audit issue: overlap warnings do not reliably block saving, and invalid end-before-start ranges may be saved as minimum-duration blocks.

```
Inspect DailyPlannerView, Quick Add, schedule create/edit flows and shared time-validation utilities.

REQUIREMENTS
1. Use one shared schedule validation function for every creation and editing path.
2. Require valid start/end times. End must be later than start for same-day blocks. If overnight blocks are supported, implement explicit date-aware validation.
3. Reject zero-length and negative-duration blocks. Never silently convert invalid ranges into 15-minute blocks.
4. Detect overlap with other blocks on the same date. Adjacent blocks (10:00–11:00 and 11:00–12:00) should normally be allowed.
5. Block overlapping saves by default. If intentional overlap is already supported, require an explicit user override.
6. Revalidate immediately before database save, not only when inputs change.
7. Apply identical rules to create, edit, duplicate, Quick Add and drag/reschedule actions.
8. Show specific inline errors and preserve entered values after validation fails.
9. Do not alter unrelated planner styling or layout.

TESTS
- End before start: blocked.
- Equal start/end: blocked.
- Valid non-overlapping block: saved.
- Adjacent blocks: accepted according to documented rule.
- Overlap: blocked or explicit override required.
- Editing a block into overlap: handled.
- Quick Add uses same validation.
- Saved blocks persist after refresh.
```

PHASE 5 — Analytics calculation correctness

Audit issue: six-month/yearly habit calculations can use completed occurrences as the denominator. Partial completion may be rounded down before aggregation. Task analytics need clear date semantics.

```
Inspect AnalyticsView and shared habit/task/time aggregation utilities. Correct underlying metric
definitions; do not patch only the visible chart.

HABITS
1. Denominator = actual habit opportunities scheduled during the selected period, based on frequency, active
dates, weekdays and custom-day rules.
2. Never use completed occurrences as the denominator or force denominator to one.
3. Exclude dates before activation and after end/archive according to historical-data policy.
4. Preserve fractional partial-completion values through aggregation; do not floor/round daily values
prematurely.
5. If there are zero scheduled opportunities, show “No scheduled occurrences”, not 0% or 100%.
6. Keep streak rules separate from completion-rate rules and document how partial completion affects each.

TASKS
7. Separate tasks due in a period from tasks completed in a period; label them clearly.
8. Use completion timestamps for completed-task trends and due dates for due-task performance.
9. Do not silently exclude tasks without due dates from all task metrics.

CONSISTENCY
10. Centralize calculations so Dashboard and Analytics cannot report conflicting values for the same metric.
11. Handle timezone/date-only values consistently.
12. Do not fabricate missing historical data.

TESTS
- 10 completed / 100 scheduled = 10%.
- 1 completed / 1 scheduled = 100%.
- Zero opportunities = empty/no-opportunity state.
- Partial completions aggregate without premature rounding.
- Part-period active habit uses only eligible dates.
- Custom weekday habit denominator follows its schedule.
- Due-task and completed-task metrics are distinct.
- Verify daily, weekly, monthly, six-month and yearly boundaries.
```

PHASE 6 — Time Tracker manual-entry and session integrity

Audit issue: manual-entry state contains relationship fields, but the form does not expose all corresponding selectors. Session edits/deletes must keep derived totals consistent.

```
Inspect TimeTrackerView, timer state, manual-entry form, session database operations and task/goal
aggregation.

REQUIREMENTS
1. Add minimal controls in the existing manual-entry form to select an associated task, habit and/or goal,
using existing records and visual styles.
2. Keep links optional where the model permits; clearly show unlinked entries.
3. Validate date, start/end or duration. Reject negative, zero or invalid durations.
4. Distinguish manual entries from timer-generated sessions if that distinction is not already stored.
5. Prevent accidental loss/replacement of an active timer. Preserve existing persistence behavior and warn
before replacing an active session.
6. Ensure paused time is excluded from tracked duration.
7. Allow editing/deleting manual sessions if supported by product design; otherwise add the smallest
consistent controls.
8. On session changes, recalculate task actual duration and goal progress through the shared Phase 3
calculation service.
9. Prevent duplicate counting through both session-goal and task-goal links.
10. Preserve historical sessions when a task/habit is archived. Define safe behavior if a linked record is
deleted.
11. Show clear save, validation and error feedback.

TESTS
- Manual entry linked to task persists.
- Entry linked to goal updates progress.
- Unlinked entry does not affect unrelated goals.
- Pause/resume excludes paused time.
- Edit/delete session recalculates task totals and goal progress.
- Refresh during active timer follows intended persistence behavior.
- Invalid durations are rejected.
```

PHASE 7 — Calendar, History, Categories and Settings

Verify these modules carefully. Do not call a feature broken unless you reproduce the issue.

```
Perform focused code review and browser-level verification of Calendar, History & Reviews, category
management and Data & Settings.

CALENDAR
1. Verify month/year navigation, selected-day state and week-start preference.
2. Ensure entries link to their source task, schedule block, habit occurrence or time session where
applicable.
3. Distinguish entry types using the existing design language.
4. Show useful selected-day details and a clear empty state.
5. Verify timezone/date boundaries and archived-record behavior.

HISTORY & REVIEWS
6. Verify daily, weekly, monthly and yearly reviews can be created, saved, reopened and edited.
7. Define whether historical summaries are snapshots or live calculations. Label/document the behavior and
ensure it is consistent.
8. Verify week-start settings and period boundaries.
9. Link summary figures to underlying records where feasible.

CATEGORIES & SETTINGS
10. Verify planner, task and goal category selectors use a consistent shared category source.
11. Preserve user-created categories through refresh, backup, restore and migration.
12. Goal category dropdown already exists in the inspected source: verify persistence; do not recreate it
unnecessarily.
13. Verify settings persist after reload and invalid settings show validation.
14. Destructive actions require explicit confirmation.

For each feature report PASS (executed), FAIL (reproduced with steps), or NOT VERIFIED (could not execute).
Code inspection alone is not a browser interaction test.
```

PHASE 8 — Full regression and release-readiness audit

Run after the earlier phases. Verify the whole application without triggering a redesign.

```
Run a full regression audit of WOLI OS after the repair phases.

MODULES
Dashboard; Daily Planner; Tasks; Habits; Time Tracker; Goals/Goal Inspector; Calendar; Analytics; History &
Reviews; Data & Settings; IndexedDB persistence; backup/restore; PWA/offline behavior if supported.

TEST SCENARIOS
1. Create, edit, complete, archive and delete a task.
2. Create, edit, partially complete, complete and archive a habit.
3. Create goals using manual, task-based, habit-based and time-based tracking where supported.
4. Link, unlink and move tasks/habits between goals.
5. Record timer-based and manual actual time.
6. Edit/delete sessions and verify dependent totals recalculate.
7. Create invalid and overlapping schedule blocks and verify validation.
8. Navigate calendar dates and open source records.
9. Create and reopen reviews.
10. Export, validate and restore a backup, including an intentionally empty backup.
11. Refresh/reopen after important data operations.
12. Verify no duplicate records, stale links, phantom progress or unexpected starter-data insertion.
13. Verify desktop, tablet and mobile layouts.
14. If PWA/offline is supported, test offline reload, local persistence and service-worker updates.

ENGINEERING
- Run available TypeScript/type checks, lint and production build.
- Check browser console and IndexedDB errors.
- Verify write operations handle errors and provide user feedback.
- Verify migrations preserve existing data.
- Remove temporary debug code and test bypasses.

FINAL REPORT
Return a table: Module | Test | Result (PASS/FAIL/NOT VERIFIED) | Evidence/steps | Remaining issue.
Then list: files changed; bugs fixed/root causes; migrations/backward compatibility; tests actually executed
and commands; manual tests still required; known limitations.

Do not claim full testing or production readiness if any critical test is FAIL or NOT VERIFIED. Do not
redesign UI or add unrelated features during this phase.
```

Execution checklist

Use this checklist after each phase before moving on.

- ■ Agent inspected the implementation and traced the root cause.
- ■ Agent named the files it intended to change.
- ■ Only files relevant to this phase were changed.
- ■ Existing UI, user data, architecture and working features were preserved.
- ■ Relevant tests were added or updated.
- ■ Available checks were run and real results reported.
- ■ Unexecuted browser tests were marked NOT VERIFIED.
- ■ I manually tested the important workflow in the running app.
- ■ I exported a backup before database migrations or restore tests.

If AI Studio makes an overly broad change

```
Stop. Your last change went beyond the requested scope. Revert only unrelated changes introduced in your latest response. Keep the targeted bug fix, preserve all existing user data and UI, and show the exact diff/files affected. Do not reset the database or rewrite unrelated components.
```

If AI Studio claims a fix without proof

```
Do not just state that this is fixed. Show the exact code path and a reproducible test for the reported scenario. Run the test if the environment supports it. If you cannot execute it, mark it NOT VERIFIED and give precise manual browser steps. Do not fabricate test results.
```

Final reminder

Run these prompts sequentially. Keep a backup of the project and data before database-related work. Review the diff and test the app after every phase. The goal is a reliable WOLI OS—not a fresh redesign.